

**Yenepoya Institute of Arts, Science,
Commerce, and Management**

Project – High Level Design

on

Identity Security

Framework for Healthcare

Student Name: SUDHEESH KS

Industry Mentor: Mr. Shashank

Guided by: Ms. Divya N Shetty

-Table of Contents

No.	Section Title	Page
1	Introduction	4
1.1	Scope of the Document	4
1.2	Intended Audience	4
1.3	System Overview	5
2	System Design	5
2.1	Application Design	5
2.2	Process Flow.	6
2.3	Information Flow	8
2.4	Components Design	8
2.5	Key Design Considerations	9
2.6	API Catalogue.	10
3	Data Design.	12
3.1	Data Model	12
3.2	Data Access Mechanism	12
3.3	Data Retention Policies	14
3.4	Data Migration	14
4	Interfaces	14
5	State and Session Management	15
6	Caching	15
7	Non-Functional Requirements	16

7.1	Security Aspects	16
7.2	Performance Aspects	17
8	References	18

1. Introduction

The Identity Security Framework for Healthcare is a secure web-based application designed to manage healthcare operations and user identities in a hospital environment. It supports key functions such as patient management, employee management, lab tests, prescriptions, and ward admissions. The system is developed using React for the frontend, Node.js and Express.js for the backend, and PostgreSQL for data storage. It also integrates a Python-based ML service for anomaly detection and enhanced security. The system uses a hybrid authentication approach with JWT for employees and session-based login for admins and patients. It also includes OTP verification, role-based access control, and protection against common cyber-attacks. Overall, the system provides a secure, scalable, and efficient platform for healthcare management.

1.1 Scope of the Document

This document presents the High-Level Design (HLD) of the Identity Security Framework for Healthcare System. It outlines the overall system architecture, key components, and interactions between the frontend, backend, database, and machine learning-based security services.

The document covers core functionalities such as authentication, authorization, user management, and healthcare operations. It also highlights system design considerations including scalability, security, and performance.

This document focuses on architectural design and system behaviour, and does not include low-level implementation details or code-level specifications.

1.2 Intended Audience

This document is intended for developers, system architects, academic evaluators, and students involved in the design and analysis of the system. It provides a clear understanding of the system architecture, design decisions, and overall workflow.

It also serves as a reference for project stakeholders to evaluate the system's design approach, scalability, and security considerations, and supports future development, maintenance, and enhancements.

1.3 System Overview

The system is a secure, full-stack hospital management platform designed to manage users, employees, and administrators, along with healthcare operations such as appointments, prescriptions, lab tests, and ward admissions.

It incorporates advanced security mechanisms, including session-based authentication for administrators and patients, JWT-based authentication for employees, OTP-based verification, role-based access control, and machine learning-based anomaly detection to identify suspicious activities and enhance system security.

2 System Design

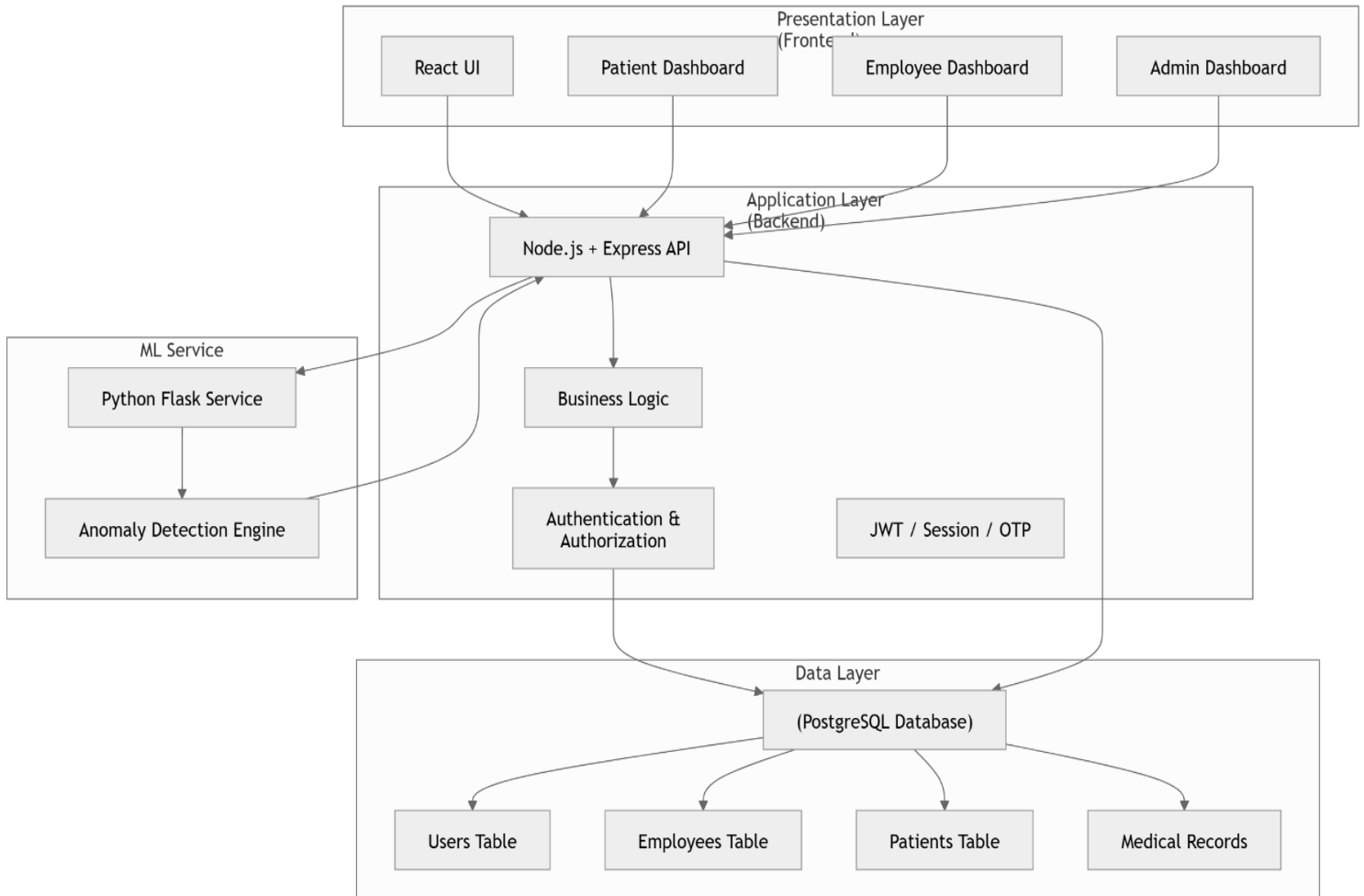
The system is designed using a modular and scalable architecture that separates responsibilities across multiple layers to ensure maintainability, security, and performance. It follows a three-tier architecture with an additional machine learning service for enhanced security.

2.1 Application Design

The system follows a three-layer architecture:

- **Presentation Layer (React Frontend):**
Provides user interfaces for patients, employees, and administrators. It handles dashboards, user interactions, and communication with backend APIs.
- **Application Layer (Node.js/Express Backend):**
Manages business logic, API routing, authentication, and authorization. It implements security features such as JWT, session management, OTP verification, and ML-based anomaly detection.

- **Data Layer (PostgreSQL + ML Service):**
Stores system data including users, patients, employees, prescriptions, lab records, and admissions. A Flask-based ML service is integrated for anomaly detection and enhanced security.



➤ Figure 2.1: Layered Architecture Diagram

2.2 Process Flow

The user initiates a request through the frontend, which is forwarded to the backend for processing. The backend validates the request and performs authentication and authorization checks. The request is then analyzed by the ML service to detect any anomalies. After successful verification, the business logic is executed, and the required data is stored or retrieved

from the database. Finally, the processed response is sent back to the frontend for display.

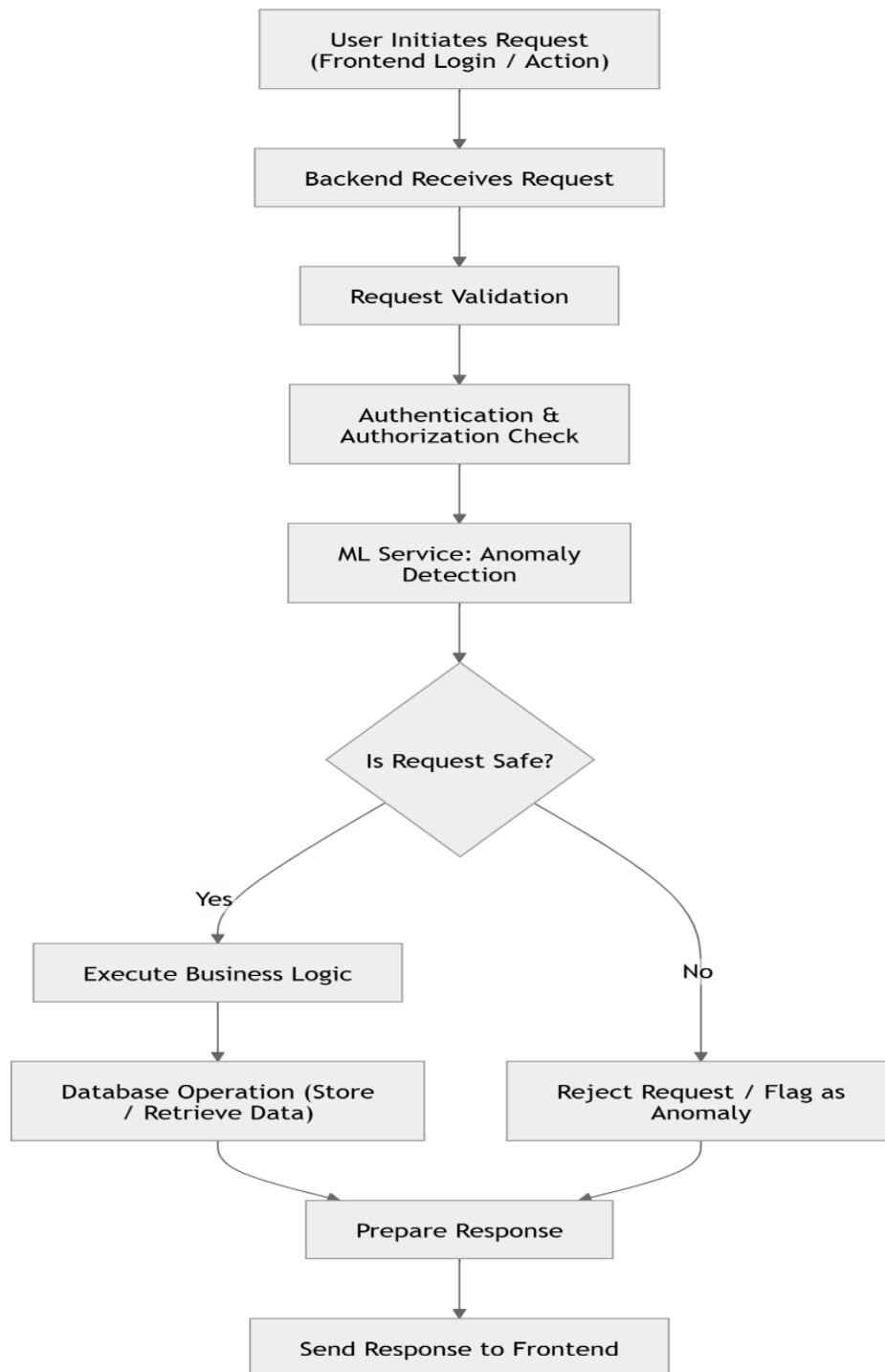


Figure 2.2: System Workflow Flowchart

2.3 Information Flow

The user interacts with the frontend, which sends HTTP requests to the backend. The backend processes the request, communicates with the database, and sends data to the ML service for anomaly checking. After processing, the response is returned to the frontend for display.

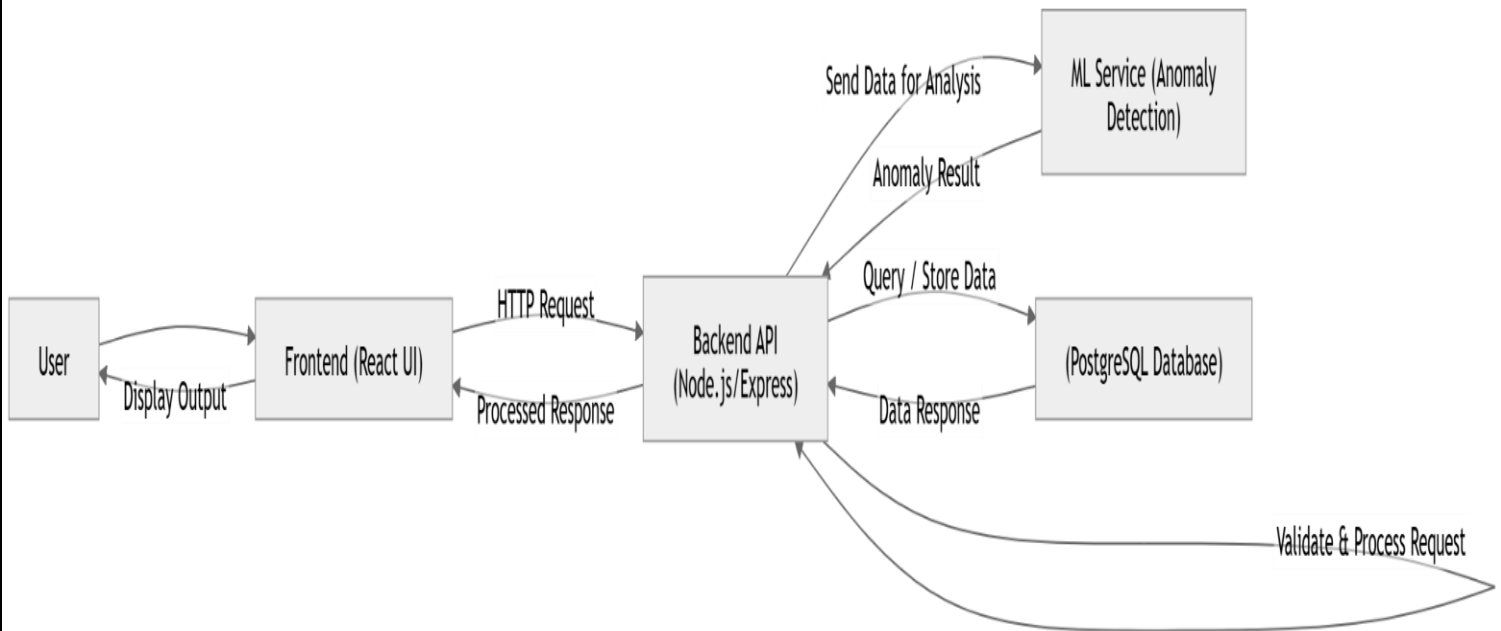


Figure 2.3: Data Flow Diagram (DFD Level 1)

2.4 Component Design

- Frontend (React): Manages user interface and communicates with APIs
- Backend (Node.js/Express): Handles business logic, authentication, and API requests
- Database (PostgreSQL): Stores and manages all persistent system data
- ML Service (Flask): Performs anomaly detection and security validation
- Email Service: Sends OTPs and system notifications

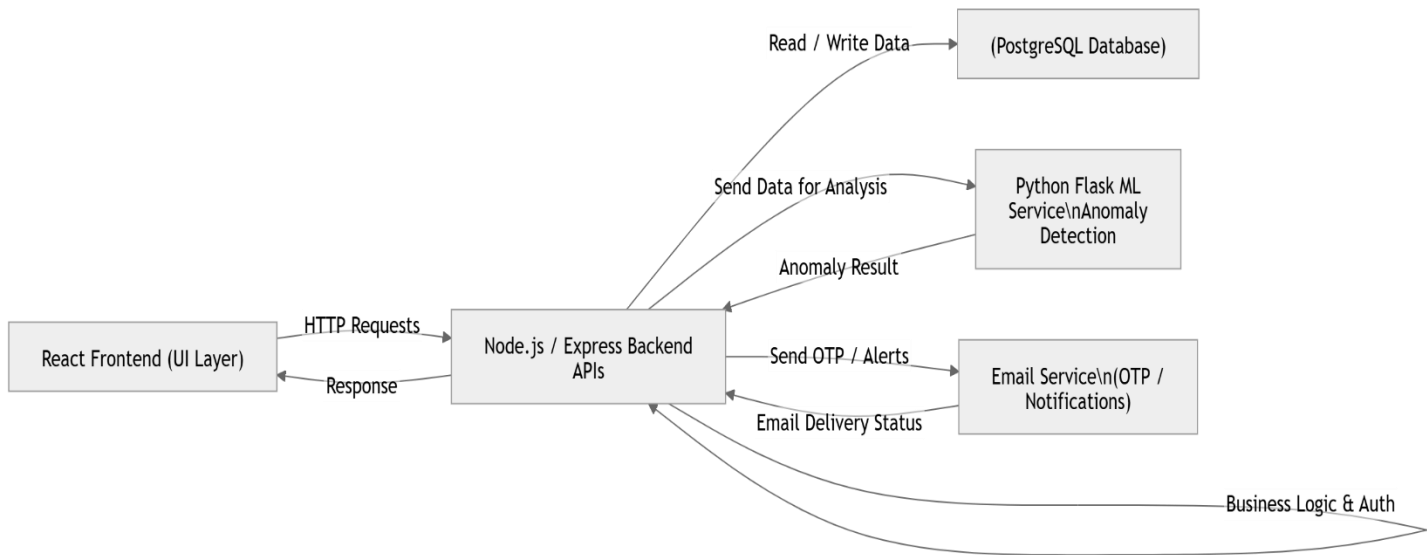


Figure 2.4: Component Diagram

2.5 Key Design Considerations

- **Security:** Implementation of strong authentication, role-based access control, OTP verification, and ML-based anomaly detection
- **Scalability:** Modular architecture with support for future microservices expansion
- **Performance:** Optimized APIs and efficient database operations
- **Maintainability:** Clear separation of concerns across system layers
- **Reliability:** Robust error handling and fallback mechanisms

2.6 API Catalogue

The system provides RESTful APIs for authentication (login, logout, token validation), employee and patient management, along with secure handling of lab tests, prescriptions, and ward admission processes.

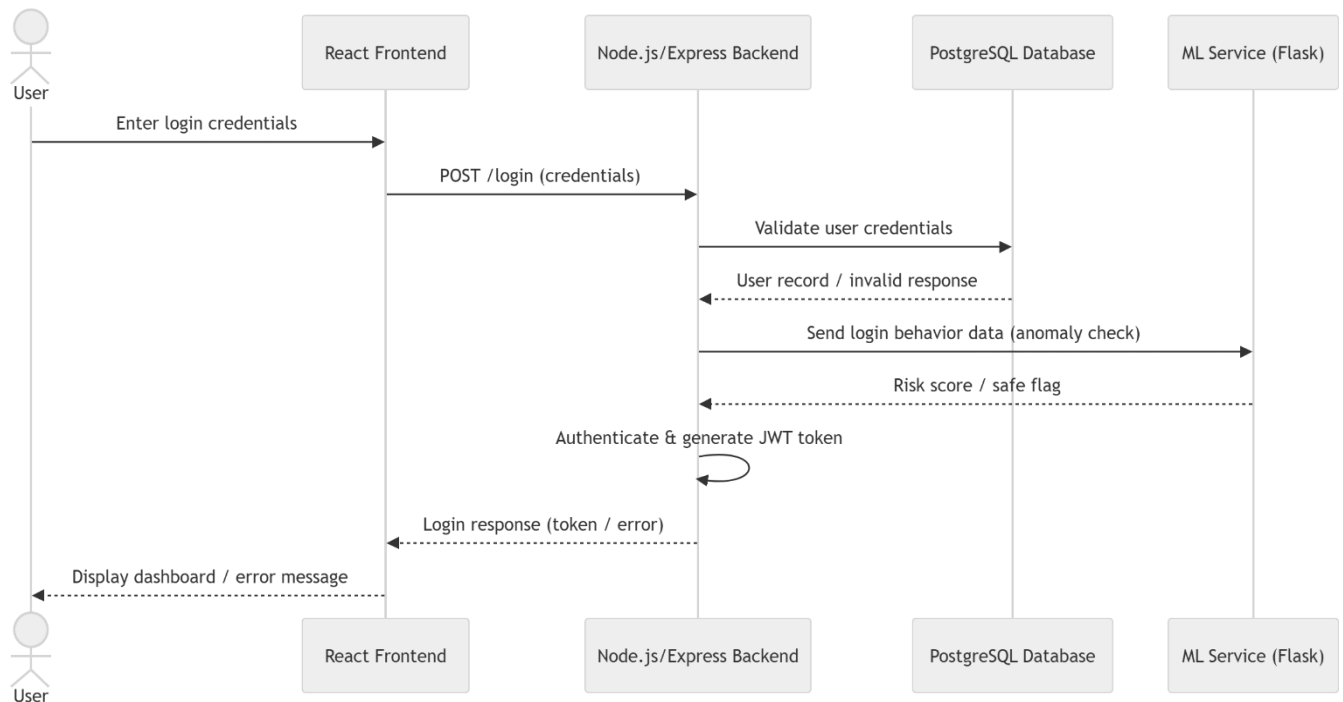


Figure 2.6: REST API Communication Flow

Module	API Endpoint	Method	Description
Home Route	/	GET	Serves the Employee Login Page
Authentication	/api/employee/login	POST	Authenticates employee and generates JWT token
Authentication	/api/admin/login	POST	Authenticates admin using session-based login
Authentication	/api/patient/login	POST	Authenticates patient

			and creates session
Authentication	/api/employee/me	GET	Retrieves logged-in employee profile
Authentication	/api/employee/send-otp	POST	Sends OTP for multi-factor authentication
Authentication	/api/employee/change-password-token	POST	Resets password using token + OTP verification
Employee Management	/api/employees	GET	Retrieves all employees
Employee Management	/api/employees	POST	Creates new employee and sends email notification
Employee Management	/api/patients/:empID	GET	Retrieves patient list based on employee role
Patient Management	/api/patient/register	POST	Registers a new patient
Patient Management	/api/patient/session	GET	Checks patient login session
Patient Management	/api/patient/logout	POST	Logs out patient and destroys session
Lab Management	/api/lab/requests/:empID	GET	Retrieves lab test requests
Lab Management	/api/lab/complete/:id	PUT	Marks lab test as completed
Lab Management	/api/lab/upload/:id	PUT	Uploads lab test report
Prescription	/api/prescriptions	POST	Creates a new prescription
Prescription	/api/prescriptions/:regno	GET	Retrieves prescriptions for a patient
Ward Management	/api/admit	POST	Admits patient to ward
Ward Management	/api/discharge/:id	PUT	Discharges patient from ward
ML Security	/api/ml/check_anomaly	POST	Detects suspicious activity using ML model

3 Data Design

The system uses a relational data model implemented using PostgreSQL to efficiently store, manage, and retrieve healthcare and security-related data. The data design ensures consistency, scalability, and secure handling of sensitive medical information.

3.1 Data Model

The data model defines the structure of the database and the relationships between different entities in the system. It includes key entities such as Employees, Patients, Lab Tests, Prescriptions, and Ward Admissions. These entities are connected through relationships like one-to-one and one-to-many to ensure efficient data organization. The model is implemented using a relational database (PostgreSQL) to maintain data consistency, integrity, and scalability.

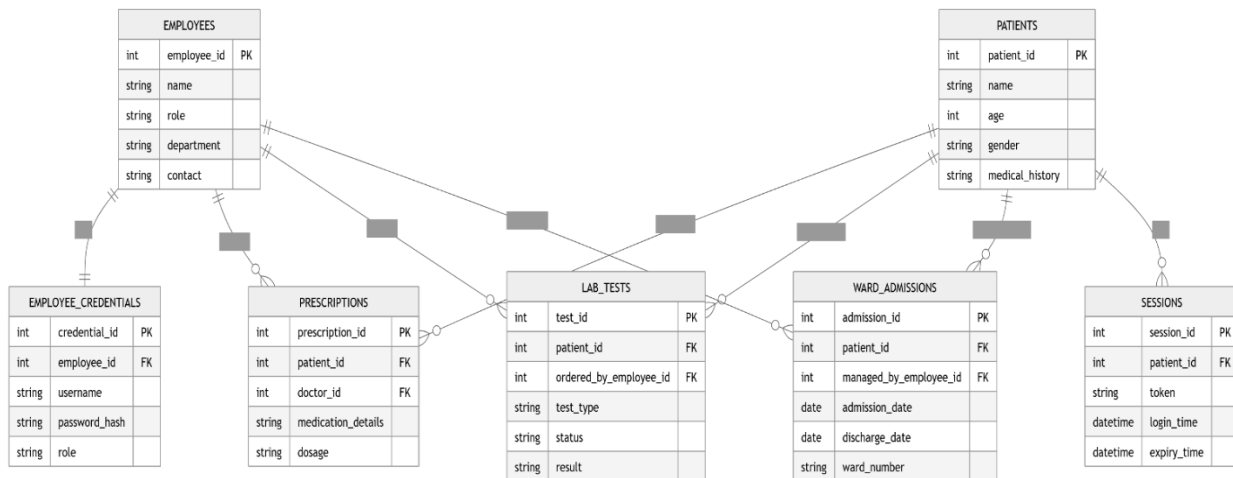


Figure 3.1: Entity Relationship (ER) Diagram

3.2 Data Access Mechanism

Data is accessed through RESTful APIs developed using Node.js and Express.js. The backend securely communicates with the PostgreSQL database using parameterized queries to prevent SQL injection attacks and ensure safe data handling.

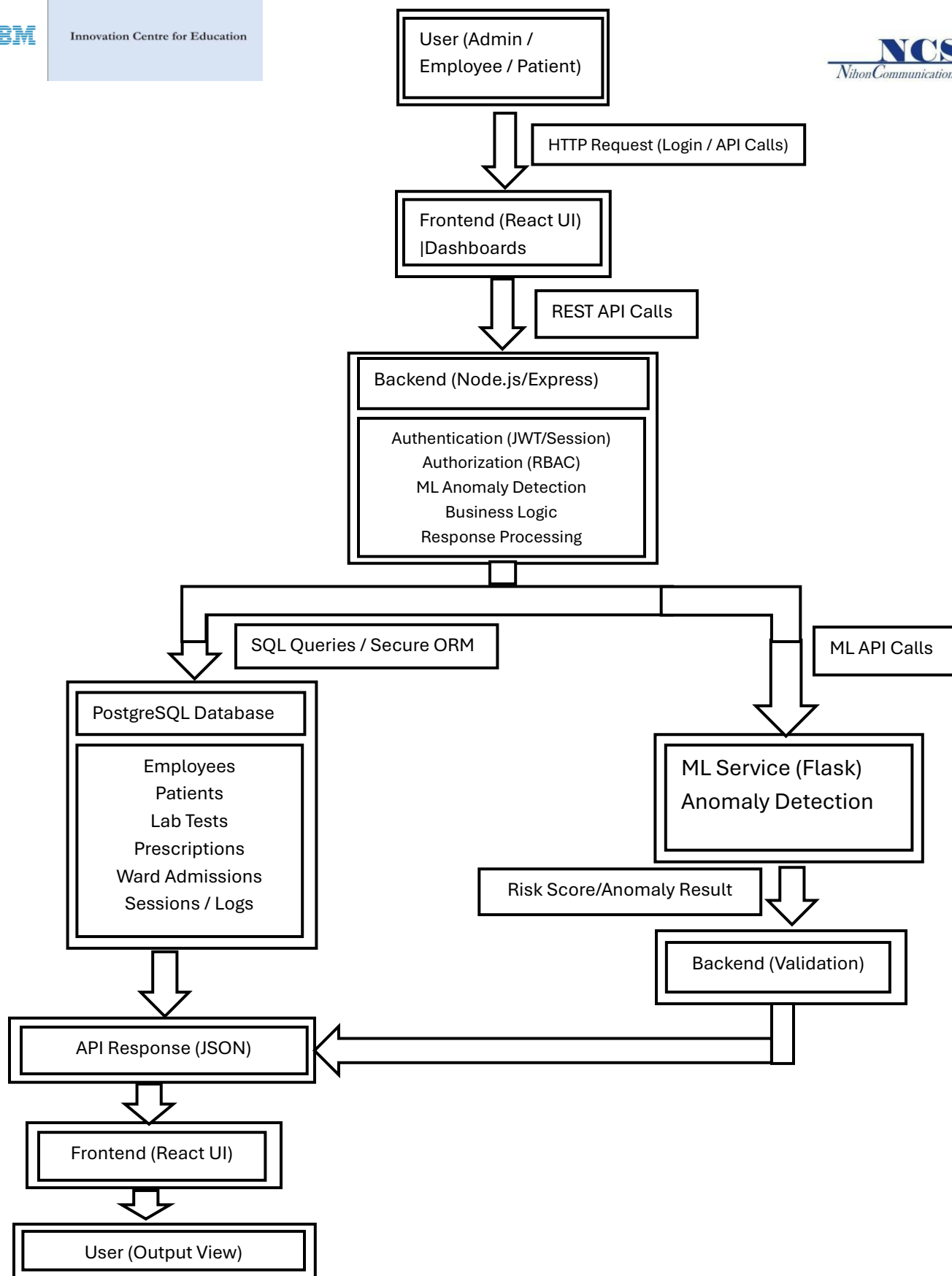


Figure 3.2: Data Access Mechanism Diagram

3.3 Data Retention Policies

- Patient and employee records are stored for long-term hospital management use
- Login sessions expire automatically after inactivity or logout
- Security logs (login attempts, anomaly detection results) are stored for monitoring
- Old or inactive records can be archived for performance optimization

3.4 Data Migration

Currently, the system uses a PostgreSQL relational database. In future enhancements, the system can be migrated to cloud-based databases such as AWS RDS or MongoDB Atlas to support scalability, high availability, and distributed access.

4 Interfaces

The system provides multiple interfaces that enable communication between users, system components, and external services. These interfaces ensure smooth interaction across frontend, backend, database, and security services.

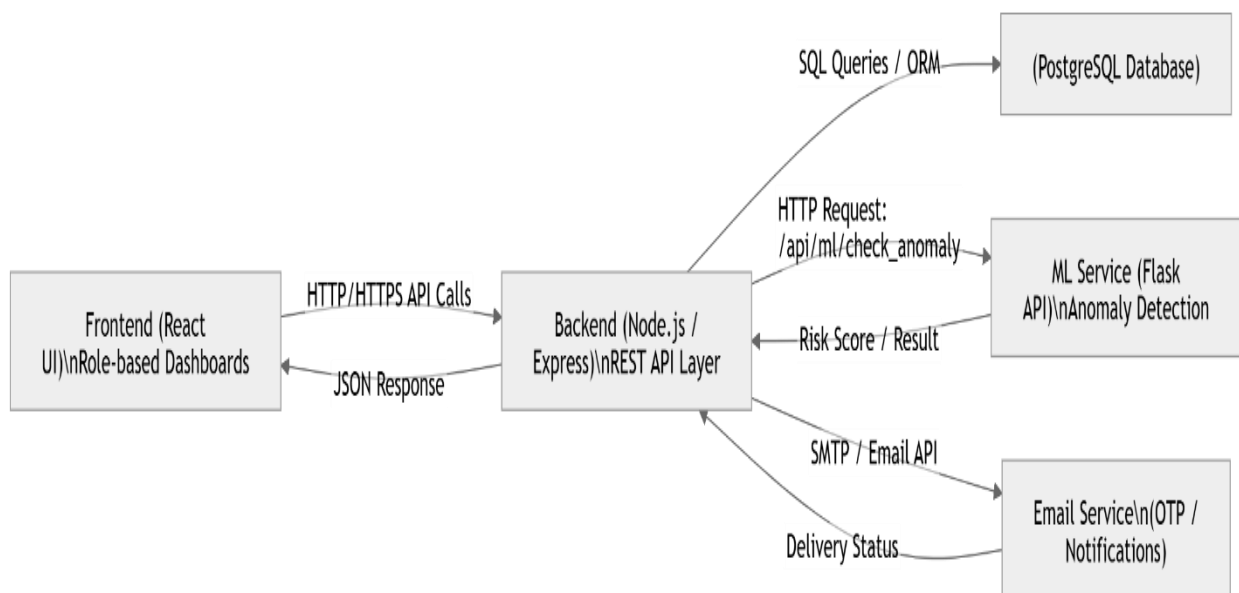


Figure 4.1: Interface Diagram

5 State and Session Management

➤ Employee Authentication Flow (JWT - Stateless)

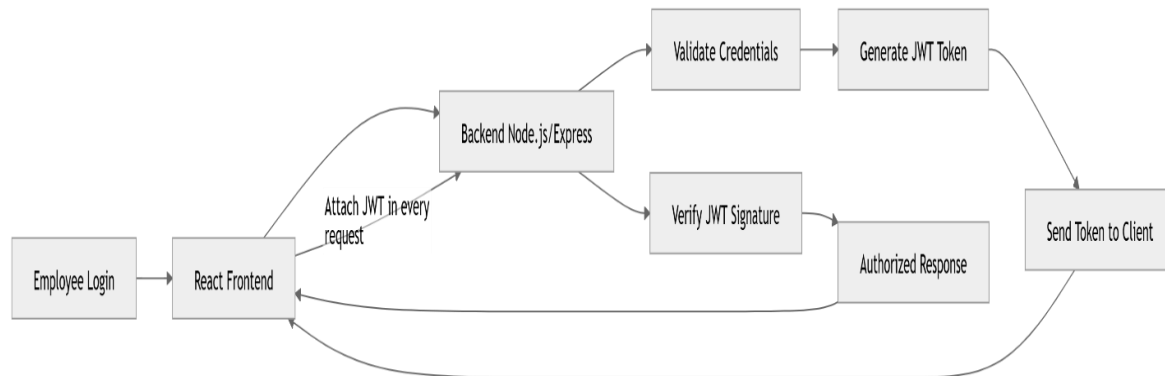


Figure 5.1: Authentication Flow Diagram – Employee (JWT)

➤ Admin / Patient Authentication Flow (Session-Based)

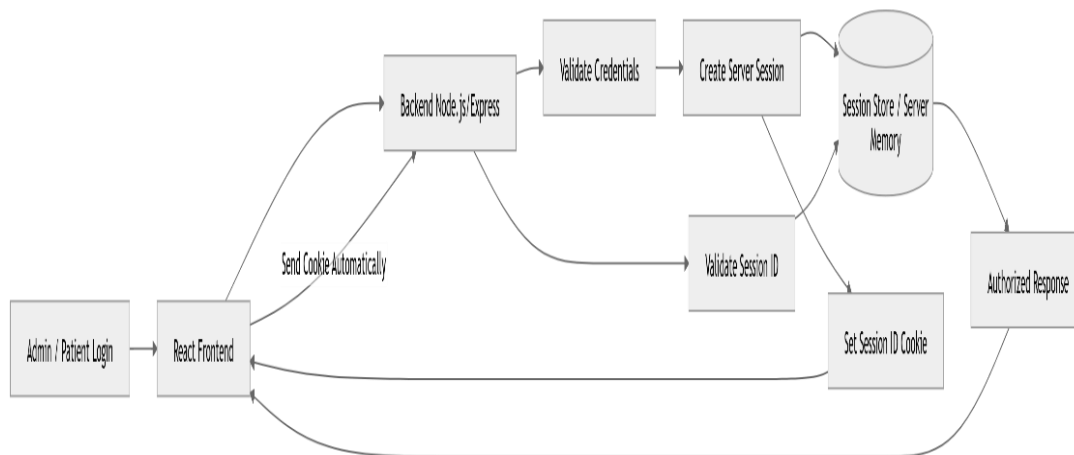


Figure 5.2: Authentication Flow Diagram – Admin & Patient (Session-Based)

6 Caching

The system currently follows a minimal caching strategy to ensure real-time accuracy of healthcare and security-related data. Since the application handles sensitive and frequently changing information such as patient records, lab results, and authentication data, most API requests are processed directly from the backend and database without persistent caching.

To improve performance, temporary in-memory caching is used in limited cases such as:

- Storing login attempts tracking data for brute-force protection
- Holding OTP and token verification data for a short duration
- Temporarily caching frequently accessed session information

The frontend may also cache some UI-level data using React state and hooks to reduce unnecessary API calls during user interaction.

In future enhancements, a dedicated caching system such as Redis can be integrated to improve response time, reduce database load, and support high scalability for large-scale hospital environments.

7 Non-Functional Requirements

The system is designed to meet essential non-functional requirements that ensure reliability, scalability, performance, and security in a healthcare environment.

7.1 Security Aspects

Security is a primary focus of the system due to the sensitive nature of healthcare data. The system implements multiple layered security mechanisms to protect user information and prevent unauthorized access.

- Authentication Security: Uses JWT-based authentication for employees and session-based authentication for admins and patients.
- Role-Based Access Control (RBAC): Ensures users can only access features permitted to their roles (doctor, nurse, lab staff, admin, patient).
- OTP Verification: Provides additional security during password reset and sensitive operations.

- **ML-Based Anomaly Detection:** Detects suspicious activities such as unusual login patterns, bot attacks, and potential SQL injection attempts.
- **Password Security:** Passwords are securely hashed using bcrypt before storage.
- **Session Security:** Sessions use HTTP-only cookies with automatic expiration to prevent hijacking.
- **API Protection:** Rate limiting and security headers (Helmet) are used to prevent brute-force and common web attacks.
- **Data Protection:** All sensitive operations are validated and sanitized before processing to prevent injection attacks.

7.2 Performance Aspects

The system is designed to provide efficient performance while handling multiple users, real-time healthcare operations, and security validations.

- **Optimized API Response Time:** RESTful APIs are designed to process requests quickly with minimal latency.
- **Efficient Database Queries:** PostgreSQL is used with structured relational design and optimized queries to reduce data retrieval time.
- **Lightweight Frontend:** React-based frontend ensures fast rendering and smooth user experience.
- **Asynchronous Processing:** Non-blocking operations in Node.js improve system throughput and allow concurrent request handling.
- **Rate Limiting:** Prevents system overload by controlling the number of requests per user/IP within a time frame.
- **ML Service Optimization:** Anomaly detection is performed only on critical authentication and sensitive requests to reduce unnecessary processing overhead.

- Minimal Caching Strategy: Temporary in-memory storage is used for OTP, sessions, and login attempts to improve response speed.
- Scalability Support: Modular architecture allows horizontal scaling of backend and ML services for higher loads.

8 References

- React Official Documentation – <https://react.dev/>
- Node.js Documentation – <https://nodejs.org/>
- Express.js Guide – <https://expressjs.com/>
- PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- Flask Documentation – <https://flask.palletsprojects.com/>
- JSON Web Token (JWT) – <https://jwt.io/introduction>
- OWASP Top 10 – <https://owasp.org/www-project-top-ten/>
- REST API Best Practices – <https://restfulapi.net/>